Github Account :

Email: git.popupalerts@gmail.com
Password: popupalerts112
Token Password: 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

PopupAlerts - SaaS Widget Platform

PopupAlerts is a full-featured SaaS (Software as a Service) platform that allows users to create, customize, and deploy a wide variety of notification widgets on their websites. This project is built with a modern tech stack, featuring a NestJS backend and a React frontend, designed for scalability, performance, and a professional user experience.

✨ Key Features

- **Multi-Widget System:** 12+ customizable widget types including informational popups, coupons, countdown timers, email collectors, social proof, and more.
- **Payment Integration:** Ready for business with both **Stripe** and **PayPal** subscription models.
- **User Management:** Complete authentication flow including registration, login, email verification, and password reset.
- **Admin Panel:** A dedicated dashboard for administrators to manage users and view platform-wide statistics.
- **Analytics Dashboard:** A personal dashboard for each user to track the performance of their widgets (views, leads, etc.).
- **Real-time Notifications:** Integrated notifications for new leads via Email, Webhook, Slack, Discord, and Telegram.
- **Visual Customization:** Users can customize the appearance of their widgets, including colors, fonts, and timing.

🚀 Tech Stack

Category	Technology
Frontend	React, TypeScript, Vite, Tailwind CSS
Backend	NestJS, TypeScript, PostgreSQL, TypeORM
Payments	Stripe, PayPal
Deployment	Ubuntu (Vultr), Nginx, PM2, Certbot (for HTTPS)

Database	PostgreSQL

🖥️ Getting Started: Local Development Setup

Follow these steps to get the project running on your local machine.

Prerequisites

- Node.js (LTS version recommended, e.g., v20.x)
- npm (usually comes with Node.js)
- Git
- PostgreSQL installed and running locally.
- A code editor like VS Code.

Installation Steps

1. Clone the Repository

git clone
https://github.com/gitpopupalerts/popupalerts.git
cd popupalerts

2. Backend Setup (popupalerts-backend)

a. Navigate to the backend directory:
bash cd popupalerts-backend
b. Install dependencies:
bash npm install
c. **Create and configure the .env file:** Create a new file named .env in this directory and fill it with your local credentials.

```.env
# Local Database Credentials
DATABASE_HOST=localhost
DATABASE_PORT=5432
DATABASE_USER=postgres
DATABASE_PASSWORD=your_local_postgres_password
DATABASE_NAME=popupalerts_db

# JWT Secret Key (generate a long random string)
JWT_SECRET=your_super_long_and_random_jwt_secret_key
```

```
# Test Keys from Stripe & PayPal Dashboards
STRIPE_SECRET_KEY=sk_test_...
STRIPE_WEBHOOK_SECRET=whsec_...
PAYPAL_CLIENT_ID=...
PAYPAL_CLIENT_SECRET=...
```

d. Setup Local Database: Open psql or your preferred database tool and run:
sql CREATE DATABASE popupalerts_db;
e. Run the Backend Server:
bash npm run start:dev
The backend will be running at http://localhost:3000. The first time it runs, synchronize: true (in app.module.ts for development) will create all the necessary tables.
3. Frontend Setup (popupalerts-frontend)

a. Open a new terminal and navigate to the frontend directory:
bash cd popupalerts-frontend
b. Install dependencies:
bash npm install
c. (Optional) Environment Variables: If you need to use environment variables in the frontend (like the PayPal Client ID), create a .env file in this directory.
.env VITE_PAYPAL_CLIENT_ID=...
d. Run the Frontend Server:
bash npm run dev
The frontend development server will be running at http://localhost:5173.
4. Accessing the App
You can now open http://localhost:5173 in your browser to use the application locally.

☁️ Deployment to Production (Vultr / Ubuntu)**

This guide provides the steps to deploy the application to a production Ubuntu server.

1. Server Prerequisites

a. **Create a non-root user with sudo privileges.**

b. Install necessary software:
```bash
sudo apt-get update
sudo apt-get install -y git nginx postgresql postgresql-contrib
  # Install Node.js (LTS)
  curl -fsSL [https://deb.nodesource.com/setup_lts.x](https://deb.nodesource.com/setup_lts.x)
| sudo -E bash -
  sudo apt-get install -y nodejs
```

```
# Install PM2 (Process Manager)
sudo npm install pm2 -g
```

2. Production Database Setup

a. Login to PostgreSQL:
bash sudo -u postgres psql
b. Create the database and user:
sql CREATE DATABASE popupalerts_prod; CREATE USER popupalerts_user WITH ENCRYPTED PASSWORD 'your_strong_password'; GRANT ALL PRIVILEGES ON DATABASE popupalerts_prod TO popupalerts_user; GRANT CREATE ON SCHEMA public TO popupalerts_user; -- Crucial for migrations \q

3. Deploy Backend

a. Clone the repository:
bash git clone https://github.com/gitpopupalerts/popupalerts.git cd popupalerts/popupalerts-backend
b. Install dependencies:
bash npm install
c. Create the PM2 Ecosystem file: Create ecosystem.config.js on the server:
javascript // ecosystem.config.js module.exports = { apps: [{ name: 'popupalerts-api', script: 'dist/src/main.js', env: { NODE_ENV: 'production', DATABASE_HOST: 'localhost', DATABASE_PORT: 5432, DATABASE_USER: 'popupalerts_user', DATABASE_PASSWORD: 'your_strong_password', DATABASE_NAME: 'popupalerts_prod', JWT_SECRET: 'your_production_jwt_secret', STRIPE_SECRET_KEY: 'sk_live_...', // Use LIVE keys for production STRIPE_WEBHOOK_SECRET: 'whsec_...', PAYPAL_CLIENT_ID: '...', PAYPAL_CLIENT_SECRET: '...', FRONTEND_URL: 'https://yourdomain.com', }, },], };
d. Build the application:
bash npm run build
e. Run the database migration: This will create all the tables.
bash npm run typeorm -- migration:run
f. Start the application with PM2:
bash pm2 start ecosystem.config.js pm2 save pm2 startup
(Follow the instruction from pm2 startup to enable it on boot).

4. Deploy Frontend & Configure Nginx

a. **Update API URL:** On your **local machine**, update popupalerts-frontend/src/api/axios.ts and other files to point to your production domain (https://yourdomain.com/api). Commit and push this change.

b. On the server, pull the latest changes:
bash cd ~/popupalerts git pull origin main
c. Build the frontend:
bash cd popupalerts-frontend npm install npm run build
d. Move files to Nginx directory:
bash sudo mkdir -p /var/www/html/popupalerts sudo cp -r ~/popupalerts/popupalerts-frontend/dist/* /var/www/html/popupalerts/ sudo chown -R www-data:www-data /var/www/html/popupalerts
e. Configure Nginx: Create a new config file sudo nano /etc/nginx/sites-available/popupalerts and paste the following (replace yourdomain.com):

```nginx
server {
listen 80;
server_name [tautan mencurigakan telah dihapus] [tautan mencurigakan telah dihapus];
    root /var/www/html/popupalerts;
    index index.html;

    location / {
        try_files $uri /index.html;
    }

    location /api/ {
        proxy_pass http://localhost:3000/;
        # ... (other proxy headers)
    }
}
```

f. Enable the site:
bash sudo ln -s /etc/nginx/sites-available/popupalerts /etc/nginx/sites-enabled/ sudo rm /etc/nginx/sites-enabled/default sudo nginx -t sudo systemctl restart nginx

5. Enable HTTPS with Certbot

a. Install Certbot:
bash sudo apt-get update sudo apt-get install -y certbot python3-certbot-nginx
b. Run Certbot: The --nginx flag will automate the configuration.

```bash
sudo certbot --nginx
```